

Lesson 6 — Intro to Prisma ORM

Node.js/Express

Game Plan

- Warm-Up
- What Is an ORM?
- Why Prisma?
- The Prisma Schema
- CRUD with Prisma
- Error Handling
- Introspection vs. Migration
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. On a scale of 1–5, how comfortable did you feel writing raw SQL last week?
2. What was the most annoying part of using `pool.query()` ?

The Pain of Raw SQL

```
const result = await pool.query(  
  `INSERT INTO tasks (title, is_completed, user_id)  
  VALUES ($1, $2, $3) RETURNING *`,  
  [title, false, userId]  
);  
const task = result.rows[0];
```

It works — but:

- No autocomplete for field names
- Easy to get column names wrong
- Schema changes mean rewriting queries everywhere

What Is an ORM?

An **ORM** (Object-Relational Mapper) lets you work with the database using JavaScript objects instead of SQL strings.

Instead of:

```
pool.query("SELECT * FROM users WHERE email = $1", [email])
```

You write:

```
const user = await prisma.user.findUnique({ where: { email } });
```

Under the hood, Prisma generates the SQL for you.

Prisma's Three Tools

1. **Prisma Schema** — define your data models in one file
2. **Prisma Client** — auto-generated, type-safe query methods
3. **Prisma Migrate** — manages database schema changes over time

```
schema.prisma  →  npx prisma migrate dev  →  Database updated  
                 npx prisma generate      →  Client updated
```

The Prisma Schema

```
model User {  
  id          Int      @id @default(autoincrement())  
  email       String   @unique @db.VarChar(255)  
  name        String   @db.VarChar(30)  
  hashedPassword String @db.VarChar(255) @map("hashed_password")  
  createdAt   DateTime @default(now()) @map("created_at")  
  Task        Task[]  
  @@map("users")  
}
```

- `@id` — primary key
- `@unique` — unique constraint
- `@map` — maps to database column name
- `@@map` — maps to database table name

The Task Model

```
model Task {  
  id          Int      @id @default(autoincrement())  
  title       String   @db.VarChar(255)  
  isCompleted Boolean  @default(false) @map("is_completed")  
  userId      Int      @map("user_id")  
  createdAt   DateTime @default(now()) @map("created_at")  
  User        User     @relation(fields: [userId], references: [id])  
  
  @@map("tasks")  
}
```

`@relation` defines the foreign key relationship — Prisma handles the JOIN for you.

Setting Up Prisma

```
npm install prisma @prisma/client  
npx prisma init
```

This creates:

- `prisma/schema.prisma` — your schema file
- `.env` — with `DATABASE_URL` placeholder

Point `DATABASE_URL` to your PostgreSQL database.

Two Paths to Your Schema

Introspection — existing database, generate schema from it:

```
npx prisma db pull  
npx prisma generate
```

Migration — write schema first, create tables from it:

```
npx prisma migrate dev --name init
```

In the assignment, you'll do **both**.

Using the Prisma Client

```
// db/prisma.js
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();
module.exports = prisma;
```

```
// In a controller
const prisma = require("../db/prisma");

const user = await prisma.user.findUnique({
  where: { email: "alex@test.com" }
});
```

One shared `prisma` instance for the whole app.

Prisma CRUD Methods

SQL	Prisma
INSERT	<code>prisma.model.create({ data: {...} })</code>
SELECT one	<code>prisma.model.findUnique({ where: {...} })</code>
SELECT many	<code>prisma.model.findMany({ where: {...} })</code>
UPDATE	<code>prisma.model.update({ where: {...}, data: {...} })</code>
DELETE	<code>prisma.model.delete({ where: {...} })</code>

Create

```
const newTask = await prisma.task.create({  
  data: {  
    title: "Buy groceries",  
    userId: 1,  
    isCompleted: false,  
  },  
});
```

Returns the created record. The `id` and `createdAt` are set automatically.

Find

```
// Find one by unique field
const user = await prisma.user.findUnique({
  where: { email: "alex@test.com" },
});

// Find many with a filter
const tasks = await prisma.task.findMany({
  where: { userId: 1, isCompleted: false },
  orderBy: { createdAt: "desc" },
});
```

`findMany` returns an array (empty if no matches).

`findUnique` returns `null` if not found.

Selective Fields with `select`

```
const user = await prisma.user.findUnique({  
  where: { id: 1 },  
  select: {  
    id: true,  
    name: true,  
    email: true,  
    // hashedPassword is excluded  
  },  
});
```

Use `select` to avoid returning sensitive data like passwords.

Prisma Error Codes

Common codes to handle in your controllers:

Code	Meaning	Status
P2002	Unique constraint (duplicate email)	400
P2025	Record not found (update/delete)	404
P2003	Foreign key violation	400

```
} catch (err) {  
  if (err.code === "P2002") {  
    return res.status(400).json({ message: "Email already in use." });  
  }  
  next(err); // pass to global error handler  
}
```


We Do — Translate This

Convert this `pg` query to Prisma:

```
const result = await pool.query(  
  "SELECT id, title, is_completed FROM tasks WHERE user_id = $1",  
  [userId]  
);  
const tasks = result.rows;
```

We Do — Handle an Error

What should happen if someone tries to update a task that doesn't exist?

```
const updated = await prisma.task.update({  
  where: { id: 999 }, // doesn't exist  
  data: { isCompleted: true },  
});
```

- What error does Prisma throw?
- What HTTP status should you return?
- Where do you catch it?

You Do (5 min)

Write a Prisma call to:

1. Find a task by `id` **and** verify it belongs to the current user
2. Return `null` if not found

```
async function getTaskForUser(taskId, userId) {  
  // your code  
}
```

Hint: Use `findFirst` with a `where` clause that includes both `id` and `userId`.

Assignment Preview

You'll convert your app from `pg` to Prisma:

1. Set up Prisma (introspect existing DB, then migrate)
2. Create `db/prisma.js` with a shared `PrismaClient`
3. Replace `pool.query()` calls in **user** and **task** controllers
4. Use `select` to exclude `hashedPassword` from responses
5. Handle Prisma-specific errors (P2002, P2025)

The API behavior stays the same — Postman tests should still pass!

Wrap-Up

In chat:

1. What's the difference between `findUnique` and `findFirst` ?
2. Why do we use a shared `PrismaClient` instance instead of creating one per request?
3. What Prisma error code means "record not found"?

Confidence Check

On a scale of 1–5:

How confident do you feel swapping from `pg` to Prisma this week?

Resources

- <https://www.prisma.io/docs/orm/prisma-client/queries/crud>
- <https://www.prisma.io/docs/orm/reference/error-reference>
- Ask questions in Slack

Closing

This week:

Prisma schema, CRUD operations, and error handling.

Next week:

Advanced Prisma — eager loading, pagination, transactions, and raw SQL for complex queries.

See you then!